A^3-CodGen: A Repository-Level Code Generation Framework for Code Reuse With Local-Aware, Global-Aware, and Third-Party-Library-Aware

Dianshu Liao, Shidong Pan, Xiaoyu Sun, Xiaoxue Ren, Qing Huang, Zhenchang Xing, Huan Jin, and Qinying Li

***Abstract*—LLM-based code generation tools are essential to help developers in the software development process. Existing tools often disconnect with the working context, i.e., the code repository, causing the generated code to be not similar to human developers. In this paper, we propose a novel code generation framework, dubbed A^3-CodGen, to harness information within the code repository to generate code with fewer potential logical errors, code redundancy, and library-induced compatibility issues. We identify three types of representative information for the code repository: local-aware information from the current code file, global-aware information from other code files, and third-party-library information. Results demonstrate that by adopting the A^3-CodGen framework, we successfully extract, fuse, and feed code repository information into the LLM, generating more accurate, efficient, and highly reusable code. The effectiveness of our framework is further underscored by generating code with a higher reuse rate, compared to human developers. This research contributes significantly to the field of code generation, providing developers with a more powerful tool to address the evolving demands in software development in practice.**

***Index Terms*—Code generation, repository knowledge mining, code reuse, retrieval-augmented generation.**

Received 3 March 2024; revised 30 September 2024; accepted 13 October 2024. Date of publication 24 October 2024; date of current version 12 December 2024. This work was supported in part by the National Nature Science Foundation of China under Grant 62302437, Grant 62262031, and Grant 62367003, in part by the Distinguished Youth Fund Project of the Natural Science Foundation of Jiangxi Province under Grant 20242BAB23011, in part by Gan Po Talent Support Program-Academic and Technical Leaders Training Program in Major Disciplines of Jiangxi Province under Grant 20243BCE51007. Recommended for acceptance by B. Adams. *(Corresponding author: Qing Huang.)*

Dianshu Liao and Xiaoyu Sun are with the School of Computing, Australian National University, Canberra, ACT 2601, Australia.

Shidong Pan and Zhenchang Xing are with CSIRO's Data61, Sydney, NSW 2015, Australia, and also with the School of Computing, Australian National University, Canberra, ACT 2601, Australia.

Xiaoxue Ren is with the State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou 310027, China, and also with Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou 310027, China.

Qing Huang is with the School of Computer Information Engineering, Jiangxi Normal University, Jiangxi 330022, China (e-mail: qh@jxnu.edu.cn).

Huan Jin and Qinying Li are with Jiangxi University of Technology, Jiangxi 330022, China.

Digital Object Identifier 10.1109/TSE.2024.3486195

I. Introduction

In the realm of Software Engineering, the advent of Large Language Models (LLMs) has revolutionized various tasks, including code generation [1], [2], [3], [4], [5]. The automatic generation of code by these models significantly reduces development time and effort [6]. Specifically, in the process of evolving software projects, software engineering developers are frequently tasked to add new features and functionality [7]. LLMs such as ChatGPT [8], GitHub Copilot [9], Codex [10] are capable of automatically generating code based on the developers' requirements (e.g., natural language descriptions of functions and function definitions).

A significant gap still persists between the code generated by LLMs and that crafted by human developers. When adding new functionality to a specific module, developers already had sufficient knowledge about the current task and the entire code repository. Also, according to a previous study [11], developers often expect that code generation tools could be aware of more context/knowledge by using specific frameworks/libraries [12]. In addition, they hope the generated code maximum utilizes specific third-party libraries (e.g., invocation of an API in a library) [13], [14], [15]. Above all, in contrast to straightforward LLM-based code generation, developers especially consider three primary categories of information:

❶ **Local Information**: This category encompasses information within the current working context, including defined function signatures and variables. It also covers the relative file paths of the module within the project, such as the module's Fully Qualified Names (FQNs), like *unstructured.documents.html*. Local information is crucial to ensuring the generated code utilizes accurate variables and operates harmoniously with the other functions in the project.

❷ **Global Information**: This category involves functions from other code files within the same repository, accessible via fully qualified import API names. A previous research highlighted that only around 30% of methods operate independently in open-source projects [16]. Hence, these external functions are essential for implementing new functions without duplicating code. For example, developers import functions "from *unstructured.partition.text_type* import *is_bulleted_text*" from other modules, instead of writing a function to judge "bulleted text" from scratch.

0098-5589 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

❸ **Third-Party-Library Information**: This category includes third-party libraries pre-installed in the code repository's environment. These libraries can be accessed directly through import statements, enhancing functionality without extra dependencies.

Unfortunately, none of the existing code generation methods take the above three types of information into consideration simultaneously. As the most common code generation tool, ChatGPT [8] demonstrates commendable coding ability [17] but relies solely on user input. GitHub Copilot [9], which explicitly utilizes local information spanning the current working context, often struggles to harness the full potential of global information presented in the entire code repository.[1] When these methods disregard local and global module information, they risk omitting crucial logic during code generation [18]. For instance, they might forget to extract text from a tag before attempting to identify it as a bullet point (*is_bulleted_text*). Furthermore, neglecting third-party library information may cause the LLMs to randomly select third-party libraries and generate code based on them. If the selected libraries third-party are unsupported under the given environment, it can lead to compatibility issues, such as "*ModuleNotFoundError: No module named xxx.*"

Therefore, to encounter the insufficient information awareness issue in LLM code generation tasks, we introduce novel **$\mathbf{A^3}$-CodGen** (Local **A**ware, Global **A**ware, and Third-Party-Library **A**ware **Cod**e **Gen**eration). This framework systematically mines and utilizes external knowledge from the local module, global modules, and third-party libraries. First, our framework begins by acquiring foundational information about functions and third-party libraries within the code repository. This initial step provides a solid function base and a third-party-library base that supports subsequent information retrieval processes. Next, it conducts knowledge retrieval from the function base, the third-party library base, and the current file. These retrieved insights are systematically organized into three distinct categories of code repository-aware knowledge. Subsequently, the framework seamlessly fuses these three types of code repository-aware knowledge into a cohesive prompt. This prompt serves as the input to the foundational LLM, enabling the model to generate code that closely resembles the work of human developers.

Our experiments are driven by two primary objectives: 1) To investigate the effectiveness of three distinct types of code repository information in assisting the model with code generation for specific code repositories. 2) To assess whether our framework can efficiently extract and utilize these three types of information. We crawl and filter 29 representative Python code repositories from PyPI, assembling 13,784 functions. To ensure a diverse benchmark dataset, we conduct a sampling process, selecting representative functions from each repository to total 383 functions. We subsequently label these benchmarks and the model outputs generated under different configurations to evaluate the performance of A^3-CodGen.

[1]https://githubnext.com/projects/copilot-view/

Our experimental results confirm the efficacy of incorporating three types of code repository-awareness knowledge in aiding the foundational model to reuse code repository knowledge. The A^3-CodeGen framework excels in recognizing and utilizing these three types of code repository knowledge, leading to exceptional performance metrics in terms of local reuse, global reuse, and third-party library reuse. Thanks to its outstanding code reuse capabilities, our framework is capable of generating concise code that seamlessly integrates with the code repository's working environment. Furthermore, we identify optimal configurations for local and global information, enabling the model to effectively leverage essential knowledge while avoiding the potential pitfalls of information overload. By comparing A^3-CodGen with GitHub CoPilot, we find that while both perform similarly in local reuse, A^3-CodGen surpasses CoPilot in global reuse and third-party library reuse. We propose potential methods for improving industry products like CoPilot by incorporating our approach.

The main contributions of this paper are as follows.

- To the best of our knowledge, we are the first to conduct an extensive empirical study, systematically investigating the usefulness of local information, global information, and third-party-library information in code generation tasks.
- We introduce a novel framework, dubbed **$\mathbf{A^3}$-CodGen Framework**, effectively knowledge mining and knowledge utilization, aiding models in better understanding and fully utilizing the knowledge within code repositories.
- We construct a new benchmark dataset, namely **RepoEval**, to reflect the effectiveness on code-repository-aware code generation. Results demonstrate the validity of the three types of code repository information and show that our framework can effectively mine and utilize this knowledge.
- We open-source our replication package [19], including the dataset, source code, and experimental results, for follow-up research.

II. Motivation

A. Limitation of Existing Code Generation Approaches

Benefiting from the rapid development of Transformer structures and pre-trained LLMs in the early 2020s, code-LLMs experienced significant growth. Transformer code models, exemplified by milestone works such as CodeT5 [20], CodeBERT [21], and GraphCodeBERT [22], dominated the scene. More recently, decoder-only Transformer models have become the mainstream in generative models. Represented by Codex [10], CodeGen [23], and others like [24], [25], [26], [27], [28].

Nowadays, to Software Engineering developers, LLM-empowered code generation has become an irreplaceable assistant tool in their development cycle [29], [30]. Although the code-LLM has been developed for years, those code generation tools still play a supporting role in the actual SE development. Essentially, there are significant differences between the code generated by LLMs and that crafted by human developers. Specifically, we conclude the phenomenon in the following threefold:

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

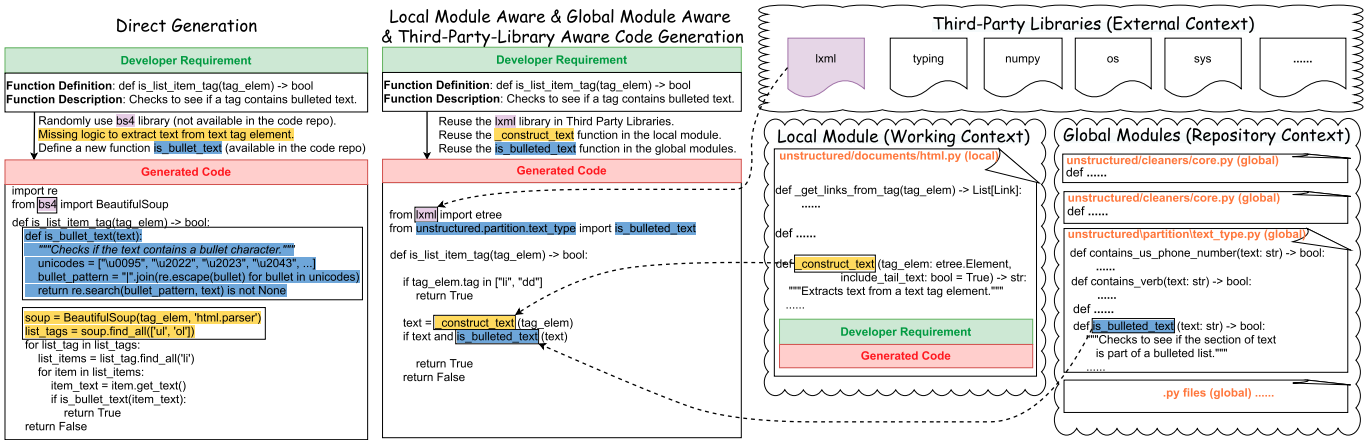

Fig. 1. Knowledge-Starved Code Generation vs. Code Repository Aware Code Generation.

- **Observation 1:** Followed by the innate features of LLMs, the content and quality of generated codes heavily depend on the human-crafted prompts.
  Discussion 1: Providing longer prompts with richer content seems a plausible solution. With the growth of LLM size, the maximum input length increases from 512 tokens for the early models [31] to 32,000 tokens for the extended GPT-4 models [32]. However, it is still impossible to feed the entire code repository and all other relevant information into the LLMs. Furthermore, studies have demonstrated that irrelevant context in prompts not only affects model efficiency and increases expenses, but also potentially compromises the model performance [33]. Therefore, to maintain the practicability and effectiveness simultaneously for code generation tasks, external context information should be extracted and selectively fed into LLMs.
- **Observation 2:** Existing methods focus on generating independent and often small-scale code units [16], [34]. According to a previous study [34], the code generated by LLMs shows lower method-level correctness. Although various benchmarks have been proposed (Details are discussed in the Related Work (Section VII)), newly generated code remains questionable.
  Discussion 2: The correctness relates to many factors, and evaluation of correctness is already challenging enough. Studies have shown that larger and more powerful foundation LLMs lead to better code generation correctness. Additionally, retrieving and reusing existing functions or APIs (which are usually verified by other human developers), as the skeleton for code generation, could also fundamentally improve the correctness. This is also the strategy that human developers commonly apply in practice.
- **Observation 3:** Some code-LLMs (e.g., GPT-4) have developed superior skills in simple algorithms such as greedy algorithm and searching strategies, but they perform poorly in complicated data structures such as tree structure and graph theory [35].
  Discussion 3: Similarly, instead of generating complex code snippets from scratch, importing functions that are already implemented can mitigate the shortcomings of advanced-level programming challenges. For example, Python developers are likely to import functions from networkx package to build a graph, not building graphs by the dictionary-like data structure.

B. Code Generation in Code Repository

During software development, programmers typically work within a specific code environment, essentially a designated code repository. This repository encompasses all project modules, functions in these modules, and third-party libraries, serving as the foundational framework for building and expanding software applications. In this context, developers, especially experienced ones, must thoroughly navigate and understand this code repository environment to ensure seamless integration of newly added code with the existing one. When it comes to LLM-based code generation techniques, solely relying on inputting developer requirements to these models fails to fully utilize the potential of LLMs in interacting with existing code repository environments. Specifically, these techniques are blind to the repository's unique naming conventions and programming styles, particularly when the completion target is relevant to cross-file context [36], [37], [38]. Instead of using a single code file, well-designed programs emphasize separating complicated functionality into independent modules, each implementing only one aspect of the desired functionality [39], [40].

As illustrated in Fig. 1, in the *Direct Generation* method, developers input their requirements directly into ChatGPT, expecting to receive code for a function that executes the task of *"checking to see if a tag contains bulleted text"*. However, the generated code, due to its lack of knowledge about local modules, fails to adequately consider a crucial factor: before determining whether a tag contains bulleted text, it's essential to first extract the text content from the provided *tag_elem*, which is a vital prerequisite. As a result, the generated code (highlighted in the yellow box) simply searches for content within the *<ul>* and *<ol>* tags. If bullet text exists outside these tags, for example, in the input "* \u0095 This is a list*

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

item.< /div>", the generated code will be unable to extract text content, thus preventing accurate detection.

Furthermore, since the model is unaware of the existence of the "*check text for bullet characters*" function in the global module, it defines a new function *is_bullet_text* (highlighted in blue) in the generated code. A large part of this code duplicates the function *is_bullet_text* in the global module *unstructured/partition/text_type.py*, resulting in code redundancy. It also brings additional concerns about the correctness of the newly generated function.

Additionally, since the model is unaware of which third-party libraries are available, it arbitrarily imports and uses the bs4 (BeautifulSoup4) library. This can result in the generated code that refers to a library that is not available, leading to the bug "*ModuleNotFoundError: No module named* ***".

These knowledge gaps highlight the limitations of existing code generation approaches, highlighting the need for more context-aware methods that can effectively interact with the code repository environment. In summary, a lack of knowledge about local and global modules can result in models generating code lacking functional constraints or containing redundancies. A lack of knowledge about third-party libraries in the code repository can lead to models using arbitrary third-party libraries, causing problems with the generated code not working. As a result, generating code in specific modules of the code repository is more than just meeting functional requirements. To generate code similar to human developers, the approach must fully understand and effectively utilize this diverse contextual knowledge to ensure that newly introduced code seamlessly coordinates with the existing code repository.

C. Bridging the Gap: Making LLMs Aware of Three Types of Repository Knowledge

Above all, in order to enable LLMs to fully utilize information spanning over the code repository environment for code generation, it is essential to provide the model with three types of knowledge: local modules, global modules, and third-party library information.

As shown in Fig. 1 *Local Module Aware & Global Module Aware & Third-Party Library Aware Code Generation*, when the developer requirement is entered, since the model is aware of the knowledge of the local module, the generated code reuses the *_construct_text* function (highlighted in yellow) in the local module to extract the text content in the text label element. In addition, because the model is aware of knowledge of the global module, the generated code reuses the *is_bulleted_text* function (highlighted in blue) from the global module *unstructured/partition/text_type.py*, thus reducing redundant code definitions. In addition, the model is aware of the existence of third-party libraries and therefore reuses the *lxml* library to ensure its availability.

In the next section, we comprehensively describe how to extract, structure, and utilize local aware, global aware, and third-party library aware information from code repositories.

III. A^3-CodGen Framework

In this section, we elaborate on our novel A^3-CodGen Framework to harness the full potential of code repository knowledge for assisting LLMs in code generation. As depicted in Fig. 2, the A^3-CodGen Framework comprises three essential phases. First, we obtain the function base and third-party library base (Section III-A), providing sufficient knowledge to support subsequent retrieval. Then, we retrieve the code repository knowledge from other code files, the third-party library repository, and the current file. After that, we organize them into 3 types of code repository aware knowledge (Section III-B). Finally, the three code repository-aware knowledge are fused into a prompt (Section III-C), serving as input to the foundation LLM, to ultimately generate codes with three essential awareness.

A. Repository Knowledge Base Construction

Code repositories contain a wealth of knowledge to exploit, but this knowledge usually exists in an unstructured form and cannot be directly utilized effectively, scattering in the code repository. Therefore, we need to perform systematic knowledge extraction to transform and compress this knowledge into a structured form in order to build a knowledge base to support subsequent knowledge utilization, such as knowledge retrieval. Thus, we perform **code repository information extraction** (Section III-A1) and **third-party library extraction** (Section III-A2) to obtain the function base and third-party-library base, respectively.

1) Code Repository Information Extraction: For a code repository, we can effortlessly obtain all code files in the entire repository by traversing all the directories within the project. Then, for each file, we extract all included functions, gathering both basic code information and semantic code knowledge. This process snowballs to extract all essential repository information.

Basic Code Knowledge Extraction. Abstract Syntax Tree (AST) is a tree representation of the source code, and studies have demonstrated its usefulness on various SE tasks [41]. In this study, we employ the Python library ast to analyze the AST for each source code file in the code repository. From the AST, we extract six essential types of basic information: 1) Comment. 2) Signature. This knowledge illustrates the parameters and returns of a function. For example, the signature of the function *_construct_text* is *def _construct_text(tag_elem, include_tail_text)->str*. 3) Function Source code. 4) Class associated with the function (if a function is a member of a class). 5) Is_Empty tag. *True* means empty function, *False* means empty label. For example, function *_construct_text* has *is_Empty* value *False*. 6) The file path of the function. This reflects the relative location of the current function in the entire code repository.

Semantic Code Knowledge Extraction. AST is criticized for its prone to long-term dependency problems. Also, simply employing AST potential weakens the capability to capture real and complex semantics [41]. Thus, we extract three types of semantic code knowledge to enhance the basic code knowledge, as follows:

1) Fully Qualified Name (FQN) of Functions. FQN indicates the function's origin in the code repository and, if

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

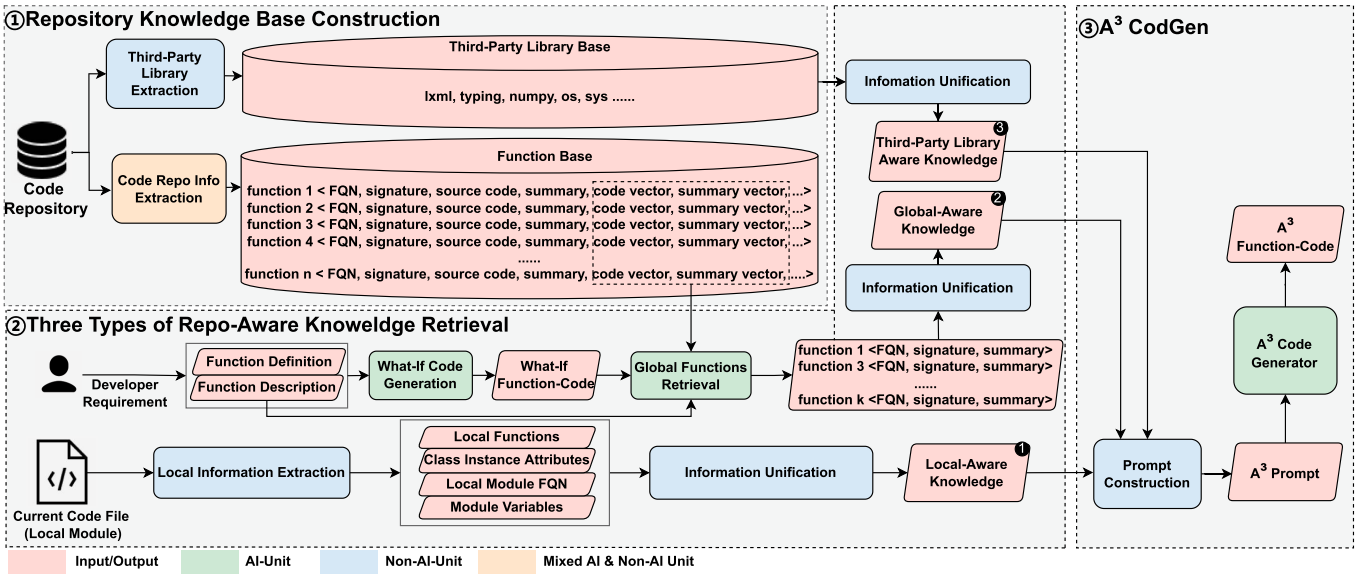

Fig. 2. A^3-CodGen Framework.

applicable, the specific class within the module. For example, in the local module of the Fig. 1, the function *_construct_text* (highlighted in yellow) has the FQN *unstructured.documents.html._construct_text*.

2) Function Summary. Not every function is elaborated by comments. Even comments exist, their quality varies; some lack clear descriptions of the function's basic functionality, while others might be over-detailed. Overly lengthy descriptions can distract from understanding the function's actual usage and behavior. Therefore, we generate a function summary to precisely describe the purpose and functionality of the function. Previous work has demonstrated that large language models (LLMs) are capable of in-context learning [42], [43], [44], where a task description and examples can enable LLMs to perform well. Thus, we use GPT-3.5-Turbo [45] as the foundation model due to its demonstrated code understanding and summarization capabilities, as well as its cost-effectiveness. The selection of the foundation model is flexible, and our method is also applicable to advanced models such as GPT-4 [32], although they come at a higher cost.

Function Summarizer Unit. To generate natural language semantic information for functions, we design a prompt as the core of the *Function Summarizer Unit*. This prompt creates summaries for each function. As shown in Fig. 3, the prompt includes a task description (highlighted in purple), five demonstration examples, and an input template. The input to this unit is the function's code (highlighted in green), and the output is the model-generated summary (highlighted in red).

3) Vector Representations. In the previous two steps, we underlined explictly select information. To ensure efficient retrieval of global functions later on, having an effective code representation that faithfully captures function semantics is essential. Hence, we employ a dual-pronged approach, conducting separate embeddings for both the source code and the function summaries. This results in the creation of *code vector* and *summary vector*, respectively.

Function Summarizer

You are an expert Python programmer.
Given a function code, summarize it with short sentences.

Example-1

```
#function code
def to_arrow_schema(schema):
    import pyarrow as pa
    fields = [pa.field(field.name, to_arrow_type(field.dataType), nullable=field.nullable) for field in schema]
    return pa.schema(fields)
#summary
    Convert a schema from Spark to Arrow
```

.....

Example-5

Please answer strictly in the examples format and do not add extra sentences!

```
#function code
    #{}#
#summary
    #{}#
```

Input: #{function code}#

Output: #{summary}#

Fig. 3. Function Summarizer Unit.

2) Third-Party Library Extraction: Third-party libraries often appear in code files, and when specific libraries are provided to a model, it likely has learned how to use them during training [46], [47]. Therefore, supplying just the library name allows the model to automatically understand how to use it for a given task. For example, we can ask the model to use the NumPy library to compute the cosine similarity, given the input *vector_a* and *vector_b*. In this case, the model can directly call functions in the NumPy library, such as *numpy.dot* and *numpy.linalg.norm*, to perform the required computations. The pre-trained model's prior knowledge and extensive large-scale training enable them to effectively utilize third-party libraries, improving the efficiency and accuracy of code generation. Therefore, we extract the import statements of third-party libraries directly from all code files via AST and saved their library names to construct the third-party library base. It is important to note that the same library can have different versions, and API call methods may vary between versions. To address this, the current repository's Python interpreter or the "pip list" command can be used to

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

determine the versions of pre-installed libraries, guiding the model in generating accurate code.

B. Three Types of Repo-Aware Knowledge Retrieval

As we discussed in Section II-A, although the maximum input length grows greatly, it is still impossible to feed the entire code repo and third-party library information to LLMs. Furthermore, irrelevant context not only affects model efficiency and increases expenses, but also potentially compromises the model performance [33].

To enable the model to fully utilize the three types of information extracted from the code repository, we conduct comprehensive knowledge mining: extracting four types of **local information** (Section III-B1), retrieving global functions from the function base using developer requirements to obtain **global-aware information** (Section III-B2), and structuring third-party library information from the **third-party library base** (Section III-B3).

1) Local-Module Knowledge Mining: Within the user's working context, referred to as the local module, four types of local-module information can be encountered:

- **Local functions.** Local functions encompass all functions that are defined and implemented within the local module. We extract these local functions and represent them in the form of <Fully Qualified Name (FQN), Function Summary, Function Signature>. The FQN provides details about the function's location within the module, the Function Summary reflects the natural language description of the function's purpose, and the Function Signature reveals its input and output specifications.
- **Class Instance Attributes.** Class instance attributes (or instance variables) are variables defined within a class's constructor function (*__init__* in Python). These variables are specific to instances of the class and store instance-specific attributes or state information. Some local functions may be defined within a class might involve interactions with class instance attributes, accessing or modifying them to achieve specific functionality.
- **Local Module FQN.** Local Module FQN partially reflects the structural information of the working context (local module) within the entire code repository. It helps the model understand the context of how local functions, class instance attributes, and variables within this module are used.
- **Module variables.** Local modules may define variables, some of which can directly interact with local functions, potentially supporting the functionality of newly added functions. We unify the declarations of these variables. However, if there are no variables defined within the module, we indicate this with *None*.

2) Global-Module Knowledge Mining: In addition to using local module knowledge, global module knowledge also plays a significant role in code generation. Instead of letting the model implement complicated logic steps from scratch, we expect the model can directly call functions if they are already present in other code files. To help models realize that global functions can be reused, we retrieve global functions in the code repository and organize them in a structured way. The global module knowledge mining process is divided into three phases: what-if code generation, global function retrieval, and information unification.

- **What-If Code Generator Unit:** In Section III-A1, we construct a function base derived from the code repository. To retrieve global functions from this base, we consider both textual and code semantic similarity. For code semantic similarity matching, we utilize the code generation capability of the LLM, inputting user requirements to generate a *what-if code*, which encapsulates the logic to fulfill those requirements and serves as input for code similarity matching in the Global Function Retrieval Unit. For in-context learning, we provide three representative examples. Notably, as the generated code is not the final code and only used to retrieve the global function, so completeness is not required. As shown in the experiment results in Section V-B, using function descriptions for text similarity and what-if code for code similarity retrieval can retrieve more useful global functions compared to using only function descriptions.
- **Global Function Retrieval Unit**: This unit uses function descriptions and the what-if function-code as input to retrieve reusable global functions from the function base. Specifically, we first embed *function_description* into $vector_{des}$ and compute the cosine similarity between this vector and the summary vectors of all functions in the function base. Thus, we can rank and obtain the top k $RetrievedFun_{des}$. Second, we embed the what-if function-code as a $vector_f$ and compute the cosine similarity between this vector and the code vectors of all the functions in the function base, so we can rank and obtain the top k $RetrievedFun_{code}$. Finally, we merge the sets of $RetrievedFun_{des}$ and $RetrievedFun_{des}$ to obtain the global function that is most relevant to the provided input, which is denoted as $RetrievedFun$.
- **Information Unification Unit**: For each $RetrievedFun$, we organize and display the retrieved information as <FQN, function signature, summary> in a structured format.

3) Third-Party Library Knowledge Mining: In the process of code generation, to behavior similar to human developers, it's crucial for the LLM to be aware of the third-party libraries pre-installed in the current code repository environment. Without this awareness, there's a risk of generating code that relies on third-party libraries unsupported by the current environment, rendering the code non-functional. To ensure that, we create a comprehensive list of all the third-party libraries found in the third-party library base, as extracted in section III-A2. It's important to note that we extract only the names of these libraries and do not include any of their code. This approach is supported by recent studies showing that LLMs can serve as neural knowledge bases for SE knowledge due to their pre-training on extensive source code corpora, which includes third-party library knowledge [42], [48], [49], [50]. Several works demonstrate library-oriented code generation, where LLMs generate code

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

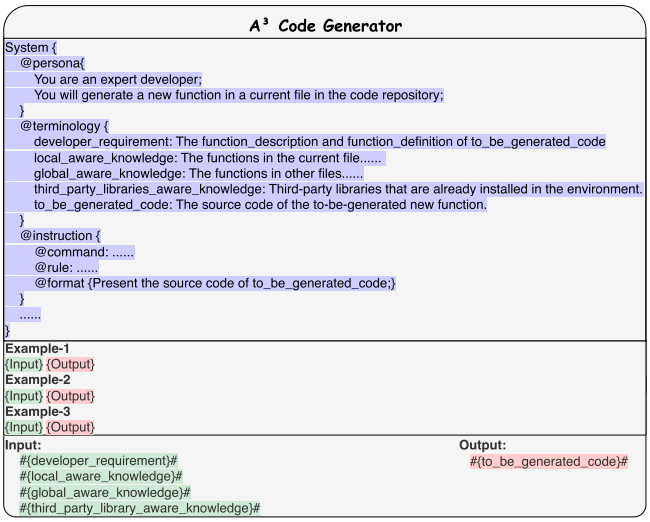

Fig. 4. A^3 Code Generator Unit.

based on specific library APIs [48], [49], [50], [51]. By constructing this list of third-party libraries, we empower the LLM to generate code that aligns with the available environment and utilizes the appropriate libraries effectively, reducing the risk of non-executable code.

C. $\mathbf{A^3}$ *Code Generation*

After section III-B, we obtain structured local-aware knowledge, global-aware knowledge, and third-party-library-aware knowledge. To ensure the model's full awareness of these three types of knowledge, we design an **$\mathbf{A^3}$ Code Generation Unit** that seamlessly integrates user requirements with the three knowledge sources to generate A^3 functional code.

The A^3 Code Generation Unit, as depicted in Fig. 4, consists of four key components: task description, examples, input, and output. In the task description (purple part), we employ a structured prompt to delineate the repository-aware code generation task. Key components of the prompt include: 1) @persona: Guiding the model to assume the role of an expert developer, providing clarity on the task. 2) @terminology: Defining crucial information in a keyword format. 3) @instruction: This section furnishes explicit instructions for executing the task:

- @command: Offering an overarching strategy for the model to follow during the task.
- @rule: Outlining the basic limitations and considerations to be observed during code generation.
- @format: Detailing the expected presentation format for the source code generated for the new function.

In the @command, we guide the model through the code generation process step by step by employing chain-of-thought [52], [53], [54].

- Firstly, in @command1, we prompt the model to contemplate the logic and steps required to fulfill the developer's request.
- Next, in @command2, we encourage the model to consider whether there is any reusable information within the local-aware knowledge and utilize it if available.
- Following that, in @command3, we guide the model to think about whether there are any reusable global functions within the global-aware knowledge and utilize them if present.
- Subsequently, in @command4, we prompt the model to explore how to leverage the third-party library-aware knowledge to interact with libraries effectively.
- Finally, in @command5, if the model cannot leverage any information, it is instructed to generate code directly based on the logic and steps it has formulated.
- Additionally, we include an @command that reiterates the importance of strictly following the steps from 1 to 5.

In the @rule, we establish three rules to govern the code generated by the model, including calling import FQN, restricting the use of specified third-party libraries, and ensuring that the generated code is not empty (as the model may occasionally generate abstract functions). We then provide three representative examples. Finally, we input the developer requirement and our 3 knowledge into Unit to get the A^3 function code (i.e., _to_be_generated_code in Fig. 4 (b)). Notably, developers only needs to input requirements, and all 3 types of repo-aware knowledge are obtained automatically.

Due to the page limitation, we have organized all prompts for our AI units (e.g., Code Summarizer, and A^3 Code Generator) in the "prompts/" folder of our artifact [19].

IV. Experiment Setting

In this section, we first outline the selection of foundation models, the dataset, and evaluation metrics for assessing our framework. We then introduce five Research Questions (RQs):

- RQ1: Does Local Module Aware Knowledge indeed help LLMs fully utilize local module knowledge during code generation, and which types are most effective in improving local awareness?
- RQ2: Is Global Module Aware Knowledge effective in assisting LLMs to utilize global module knowledge during code generation, and what is the optimal "k" value for retrieving global functions?
- RQ3: Does Third-Party Library Aware Knowledge significantly impact LLMs' awareness of available third-party libraries during code generation?
- RQ4: What is the overall performance of our A^3-CodGen framework?
- RQ5: How does A^3-CodGen framework compare to existing code generation tools like GitHub Copilot?

A. *Foundation Models*

We select GPT-3.5-Turbo-16k [45] as the default foundation LLM for this study due to its excellent performance in various SE tasks, along with its popularity and cost-efficiency. As for the embedding model, we use the *ada-embedding-002* model [55], which generates high quality sentence embeddings for effective similarity computation and retrieval operations. [56], [57]. Similar to other LLM-empowered frameworks in SE domain, the more powerful model is selected, the better performance can be achieved, generally.

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

B. Dataset Collection

While there exist some benchmark datasets designed for code generation, such as [16], [34], [58], [59], [60], they suffer from the following issues: 1) some of the existing datasets contain code repositories that are already present in the training corpora of popular LLMs, leading to an overestimation of the code repository-aware code generation capabilities [61], [62]. 2) existing benchmarks only evaluate class-level or small unit test cases, which is not suitable for our research goals. 3) Based on our literature review, none of existing code generation benchmark datasets can evaluate whether A^3-CodGen can effectively obtain, understand, and correctly reuse three distinct types of code repository information. Therefore, to address the potential risk of data leakage and accurately reflect the capability of A^3-CodGen, we created a new benchmark dataset, dubbed **RepoEval**. This decision stems from our intention to verify that the model truly grasp A^3 code generation after receiving information we provide.

1) Code Repository Collection: To construct a benchmark dataset, we follow methodologies similar to those in previous works [63], [64], [65]. As we focus on Python code repositories, it is common to collect packages from the PyPI ecosystem, as mentioned in Alfadel et al. [65].

To ensure that the chosen repositories are not part of the training data for the models used, we verify the release dates of each repository to identify their first release date. If the first release date is after the training period of the model, it means that the repository was not included in the training data. According to OpenAI's official website, the training data for GPT-3.5-Turbo-16k is cut off as of September 2021. Therefore, we select projects with the earliest release date after October 1, 2021, from PyPI. We further filter the GitHub repositories by choosing those with more than 100 stars and 100 branches, as these metrics indicate repository quality [64]. Ultimately, we identify 29 code repositories to construct our benchmark, each averaging 82.517 code files.

Then, we create 29 function bases and 29 third-party library bases for the corresponding code repositories, following the method outlined in Section III-A. The function bases from these 29 code repositories encompass a total of 13,784 functions. Out of these, 10,077 are class member functions, leaving 3,707 as module functions. Since class member functions often involve complex relationships related to inheritance and other classes, it becomes highly labor-intensive to analyze the code invocation relationships to construct the benchmark. Therefore, our benchmark dataset focuses solely on module functions. From these 3,707 module functions, we filter out functions that had comments and were non-empty, and finally, we obtain 1,533 functions.

2) Benchmark Construction: Given the comments and the definition of a function, the corresponding source code for that function serves as the standard answer to be generated. Simultaneously, we extract call information for the function to determine the extent of code reuse within it, because a higher extent of function reuse reflects the framework learns better from three awareness.

Given a function, code reuse falls into one of the three categories: 1) It could be part of locally defined functions within the local module; 2) belong to a global module; or 3) belong to a third-party library. To obtain this reuse information, we need to analyze the call relationships between functions among codes. However, existing code analysis tools like pycallgraph [66] require code compilability to accurately generate call graphs for functions. Considering that it is impractical to set up running environments for all these code repositories due to diverse requirements and potential conflicts, and given that static analysis tools like pyan [67] have relatively lower accuracy and cannot meet our needs, we resort to manual annotation to extract the call information between functions.

Manual annotation for each function is resource-intensive and time-consuming. To ensure experiment efficiency, we employ a statistical sampling method [68]. Specifically, we perform sampling on the 1,533 module functions obtained in Section IV-B1. We ensure that the estimated accuracy within the sample remains within a 0.05 error range at a 95% confidence level while ensuring functions from each code repository are included in the sample. Finally, we acquire 383 functions for subsequent experiments.

C. Metrics

To comprehensively evaluate whether the three awareness are effective, we employ the following metrics:

- **Reuse Awareness**: To assess the model's ability to correctly reuse local, global, and third-party library information when necessary and avoid reuse when unnecessary, we computed the following awareness metrics:
  - **True Positive (TP)**: The model-generated code correctly reuses local, global, or third-party library information, consistent with the benchmark code. For instance, if the model-generated code reuses local functions and the benchmark code also reuses local functions, it is considered TP in the case of local reuse, as well as in the case of global reuse and third-party library reuse.
  - **False Positive (FP)**: The model-generated code incorrectly reuses local, global, or third-party library information, where no reuse occurs in the benchmark code.
  - **False Negative (FN)**: The model-generated code fails to reuse local, global, or third-party library information, but reuse occurs in the benchmark code.
  - **True Negative (TN)**: The model-generated code correctly does not reuse local, global, or third-party library information, consistent with the benchmark code.
  - **Precision**: Calculated as $TP/(TP + FP)$, indicating the proportion of the model-generated code that is correctly reused.
  - **Recall**: Calculated as $TP/(TP + FN)$, indicating the proportion of the code that actually needed to be reused and is correctly reused by the model.

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

  - **F1**: The harmonic mean of Precision and Recall, providing an overall measure of the model's effectiveness in reusing code, calculated as $2 * (Precision * Recall)/(Precision + Recall)$.
  - **Accuracy**: Calculated as $(TP + TN)/(TP + FP + TN + FN)$, representing the percentage of cases where the model is correctly aware or unaware.
- **Reuse Correctness**: After identifying local/global/third-party library information, we assess the correctness of reuse. The metric is determined by the number of exact matches between the functions/libraries reused by the model and those in the benchmark, divided by the total number of functions/libraries reused by the model. For instance, in local reuse, if the model reuses functions $local_a$, $local_b$, and $local_c$, and the benchmark reuses functions $local_b$, $local_c$, and $local_d$, there are 2 exact matches, resulting in a correctness score of 2/3 = 0.667. It's important to note that we do not use conventional metrics like BLEU [69], ROUGE [70], or METEOR [71], as they only evaluate token-level similarity between the model-generated code and the benchmark code. These metrics do not effectively measure the model's ability to correctly reuse code components during generation.
- **Library Coverage:** This metric represents the proportion of libraries used in the generated code that are already pre-installed in the code repository's environment.
- **Lines of Code (LOC):** This metric reflects code conciseness, determined by counting lines after removing comments. Lower LOC indicates more concise code.

To validate the call relationships between the extracted functions, we recruit two developers with over five years of Python programming experience. Each function in the benchmark dataset is independently labeled by both developers. They evaluate whether the function utilizes local functions within the current code file, assigning a score of 1 if it does and 0 if it doesn't. Similarly, they assess whether the function utilizes global functions and third-party libraries, assigning scores of 1 or 0 accordingly. These scores contribute to calculating reuse awareness. For functions involving reuse, they identify all reused functions/libraries to calculate reuse correctness.

After the independent labeling, discrepancies are discussed to reach a consensus. If any disagreements remain, the first author resolves them and makes the final decision. In subsequent experiments, we use this benchmark for evaluation and annotate the model-generated code using the same method. To label the model-generated results, we recruit 4 master's students and 8 undergraduate students, each with at least 2 years of Python programming experience. After a 1-hour training session on our standardized method, tasks are evenly distributed among them. We utilize Cohen's kappa [72] to assess inter-rater agreement, with coefficients ranging from 86% to 95% for all annotated data (including the benchmark dataset and model-generated results), indicating high agreement among assessments.

V. Evaluation

A. RQ1: Does Local Module Aware Knowledge Indeed Help LLMs Fully Utilize Local Module KnowledgeDuring Code Generation, and Which Types Are Most Effective in Improving Local Awareness?

TABLE I
Comparison of the Local Reuse Awareness Results Obtained With Different Local Information

Variant	Precision	Recall	F1	Accuracy
Vallina GPT-3.5-Turbo-16k	N/A	N/A	N/A	0.721
GPT-3.5-Turbo-16k +Local Functions	0.667	0.528	0.589	0.796
GPT-3.5-Turbo-16k +Local Functions +Class Instance Attributes	0.720	**0.632**	**0.673**	**0.830**
GPT-3.5-Turbo-16k +Local Functions +Class Instance Attributes +Module Variables	0.728	0.557	0.631	0.820
GPT-3.5-Turbo-16k +Local Functions +Class Instance Attributes +Module Variables +Local Module FQN	**0.759**	0.566	0.649	0.830

1) Motivation: Local information aids LLMs in code generation by offering awareness of local contextual details. We extract and organize four types of Local-Aware Knowledge from the local module: local functions, class instance attributes, local module FQNs, and module variables. These knowledge types offer diverse perspectives on the context of the local module. Our goal is to assess whether local-aware knowledge genuinely enhances LLMs' ability to reuse local functions in code generation. Additionally, we aim to identify the optimal configuration, determining whether all four knowledge categories should be available to the model's awareness or only specific ones.

2) Experiment Methodology: We use vanilla GPT-3.5-Turbo-16k as a baseline, which does not have any local information in its input. We define four variants of local-aware methods, as shown in Table I.

- +*Local Functions* signifies the use of Local Functions knowledge from the current file context.
- +*Class Instance Attributes* indicates the use of Class Instance Attributes Knoweldge
- +*Module Variables* involves using the module Variables knowledge in the current file context.
- +*Local Module FQN* employs the fully qualified name corresponding to the current file.

We run these five different configurations on the benchmark to obtain code generation results.

3) Result Analysis: Table I shows the reuse awareness results for code generation using vanilla GPT-3.5-Turbo-16k and four types of Local-Aware Knowledge Configurations. Vanilla GPT-3.5-Turbo-16k exhibits a precision and recall of 0 as it does not recognize any local functions. Consequently, in its generated code, new functions are often defined to complete certain logic, resulting in code redundancy.

When given knowledge of *Local Functions*, the model achieves a precision of 0.667, recall of 0.528, F1 score of 0.589, and accuracy of 0.796. This is because *Local Function* knowledge allows the model to directly invoke these functions. Upon additionally providing knowledge of *Class Instance Attributes*, the model's precision, recall, and F1 score improve by 7.94%,

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

19.70%, and 14.26%, respectively, compared to providing only knowledge of Local Functions. This improvement is attributed to the provision of *Class Instance Attributes*, which enables the model to better understand how local functions interact with class variables in the project, leading to more accurate application of local functions.

However, when we introduce the *Module Variables*, the model's performance drastically declines, particularly with a 11.87% drop in precision. This may be due to the increased information load resulting from knowledge of *Module Variables*, which impacts performance. Nevertheless, the model's performance is restored and approaches the capabilities of providing knowledge of *Local Functions* and *Class Instance Attributes* when we subsequently add knowledge of Local Module FQN. This suggests the existence of subtle interactions within the knowledge configuration. When the model receives only one or two key knowledge types, its performance may be superior to when it receives more information. However, as the volume of knowledge increases to a certain extent, the model's capabilities strengthen, resulting in improved performance.

We ultimately choose the Local-Aware Knowledge setup, including knowledge of *Local Functions* and *Class Instance Attributes*. This configuration serves as default for the upcoming experiments.

Finding 1: *The Local-Aware Knowledge significantly enhances the model's ability to reuse local functions, thereby reducing code redundancy. The best configuration is the combination of Local Functions and Class Instance Attributes.*

B. RQ2: Is Global Module Aware Knowledge Effective in Assisting LLMs to Utilize Global Module Knowledge During Code Generation, and What Is the Optimal "k" Value for Retrieving Global Functions?

1) Motivation: In this RQ, our goal is to explore the effectiveness of global-aware information in aiding LLMs to reuse global functions in code generation. When using global-aware knowledge, a critical issue arises: if too few candidate functions are retrieved, the model may lack the necessary functions to generate code that correctly reuses global functions. Conversely, retrieving too many candidate functions can lead to the presence of irrelevant functions, making it difficult for the model to identify the required ones. Research shows that irrelevant context can decrease model efficiency, increase costs, and potentially compromise performance [33]. Therefore, we aim to identify the optimal top-k value for retrieving candidate functions to achieve the best global-aware configuration. Additionally, we conduct an ablation study to evaluate the effectiveness of the What-if Code Generator Unit based on the optimal k value.

2) Experiment Methodology: We employ *No Global-Aware* as the baseline and this involves providing only Local-Aware Knowledge without Global-Aware Knowledge input. To determine the optimal top-k value, we pre-define four variants: k=1, 5, 10, and 15. These values are chosen because larger projects with more complex call relationships require at least one global function. A single retrieved global function may not be sufficient, and some functions may need to call multiple global functions, so we increase the k value to 5, 10, and 15. This selection ensures we cover a range of potential requirements, balancing between too few and too many candidate functions to maintain model performance and relevance. We run the total five different configurations on the benchmark to obtain code generation results. Subsequently, we assess the generated code using the same method as in section V-A, where each result is evaluated for whether it uses global functions.

TABLE II
COMPARISON OF THE GLOBAL REUSE AWARENESS RESULTS OBTAINED FOR DIFFERENT TOP K NUMBERS OF RETRIEVED FUNCTIONS TO FIND THE BEST K VALUE

Global Functions of Different Top k	Avg. Retrieved Functions	Precision	Recall	F1	Accuracy
No Global-Aware (k = 0)	N/A	N/A	N/A	N/A	0.843
k=1	1.757	0.372	0.533	0.438	0.786
k=5	8.154	**0.518**	**0.717**	**0.601**	**0.851**
k=10	16.031	0.479	0.583	0.526	0.836
k=15	23.592	0.407	0.583	0.479	0.802

To further evaluate the effectiveness of our *What-if Code Generator Unit*, we conduct an experiment using the benchmark, focusing on test cases that call global functions (47 cases in total). We compare two retrieval methods:

- FunDes: Uses only function descriptions for text similarity retrieval.
- FunDes+FunCode: Uses function descriptions for text similarity retrieval and the What-If Code Generator to generate intermediate code snippets for code similarity retrieval.

3) Result Analysis: Table II presents the results obtained using "No Global-Aware" and four different top-k values for retrieving global functions. The *No Global-Aware* has both precision and recall as 0 because of lacking of global-aware information as we expected. Consequently, the generated code only utilizes local functions to accomplish tasks, and it resorts to defining new functions for specific logic if local functions are insufficient. When k=1, the model shows some awareness of global functions to a certain extent, underlined by the precision and recall as 0.372 and 0.533, respectively. At k=5, the model's performance significantly improves, indicating that the model now has a more comprehensive understanding of global-aware information. However, when k=10, the model's F1 score drops by 12.48%, indicating a substantial reduction in the model's ability to reuse global functions. As k rises to 15, the effectiveness of the model decreases further. This is because the overdosed amount (between 10 to 20 functions) of functions is retrieved, introducing excessive interference and hindering accurate reusing.

To further assess the impact of the number of global functions presented to the model, we compute the average number of retrieved functions, denoted as *Avg. Retrieved Functions*. We find

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

TABLE III
COMPARE THE DIFFERENCE BETWEEN HAVING THIRD-PARTY-LIBRARY AWARE AND NOT HAVING IT ON THIRD-PARTY LIBRARY REUSE AWARENESS AND LIBRARY COVERAGE

3rd-Party-Lib Awareness	Precision	Recall	F1	Accuracy	Library Coverage
No	0.716	0.429	0.536	0.715	0.875
Yes	**0.935**	**0.585**	**0.720**	**0.825**	**0.940**

that providing the model with around 8 functions yields the best performance in terms of code reuse, striking a subtle balance. Thus, we select k=5 as optimal configuration for subsequent experiments.

To further evaluate the effectiveness of our *What-if Code Generator Unit*, we conduct an experiment using the benchmark. The results showed that the *FunDes* method retrieved 41.5% of global functions, while the *FunDes+FunCode* method retrieved 60.6%, representing an almost 20% increase. This demonstrates that our *What-if Code Generator Unit* enhances the *Global Function Retrieval Unit* by providing intermediate code for code similarity metrics, thus retrieving more relevant global functions.

Finding 2: *Global-Aware Information significantly enhances the model's ability to reuse functions in other code files in the repository. The optimal configuration for retrieving global functions is k=5. The What-if Code Generator Unit helps retrieve more relevant global functions, further enhancing the Global Function Retrieval Unit.*

C. RQ3: Does Third-Party Library Aware Knowledge Significantly Impact LLMs' Awareness of Available Third-Party Libraries During Code Generation?

1) Motivation: In section III-B3, we delve into third-party-library aware information, which helps the model recognize pre-installed third-party libraries in the code repository. In this section, we aim to evaluate the effectiveness of third-party-library knowledge.

2) Experiment Methodology: To validate this point, we compare code generation between with and without third-party-library aware information. Only local-aware and global-aware information is fed into the model. Then, we run these two different configurations on the benchmark to obtain code generation results. Notably, we introduce the *Library Coverage*, the percentage of third-party libraries used in the generated code in the third-part-library base, to further elaborate the extent of reusing.

3) Result Analysis: Table III demonstrates that after incorporating Third-Party-Library Aware Knowledge, the model exhibits significant improvements in precision, recall, F1 score, and accuracy, with increases of 30.59%, 36.36%, 34.33%, and 15.38%, respectively. This enhancement can be attributed to the fact that, with Third-Party-Library Aware information, the model is suggested to use a specific subset of libraries, narrowing down its choice range and making it more likely to generate desired code. Our finding is further proven by that methods with Third-Party-Library Aware information outperform those without such knowledge in the Library Coverage metric by 7.43%. However, library coverage does not reach 100% due to occasional instances where the model obstinately believes that there are no viable libraries in the third-party repository to implement the code, which could be attributed to model hallucinations [73], [74], [75]. To mitigate these hallucinations, one potential approach is to use a reflective approach [51], [76], [77]. If the model uses a non-pre-installed library, we can inform the model that the library is not expected and prompt it to regenerate the code. This process is repeated until the model uses a pre-installed library. Although we have not systematically validated this approach due to the high labeling effort and API token costs, it represents a feasible method to mitigate model hallucinations.

Finding 3: *Third-Party-Library Aware information significantly improves the model's ability to utilize available third-party libraries, reducing risk of potential compatibility issue.*

D. RQ4: What is the Overall Performance of Our A^3-CodGen Framework?

1) Motivation: In previous sections, we have individually verified the effectiveness of local-aware, global-aware, and third-party-library-aware information. We also explored the optimal configurations. In this section, we aim to integrally validate the effectiveness of our A^3-CodGen framework from various perspectives. Therefore, we conduct an ablation study to evaluate A^3-CodGen in terms of local reuse, global reuse, and third-party reuse. Additionally, as more powerful models are released, we aim to compare the performance of A^3-CodGen across different foundation models.

2) Experiment Methodology: We first compare our A^3-CodGen with three baselines using the GPT-3.5-Turbo-16k model [45]: vanilla GPT-3.5-Turbo-16k, A^1 Generation (local-aware), and A^2 Generation (local-aware and global-aware). Then, to investigate whether performance improvements are significant with enhanced model capabilities, we use the latest LLM model, GPT-4o [78], as the foundation model for A^3-CodGen and compare its performance against GPT-3.5-Turbo-16k. We evaluate our framework using four metrics: reuse awareness, reuse correctness, library coverage, and lines of code (LOC). The F1 score serves as a comprehensive measure of reuse awareness. Notably, a lower LOC signifies more concise code.

3) Result Analysis: Upon evaluating the awareness aspect, as shown in Table IV, A^3-CodGen based on GPT-3.5-Turbo-16k demonstrates excellence in local reuse, global reuse, and third-party library reuse. This underscores the effective assistance provided by the three categories of repository-aware knowledge in enabling the model to be cognizant of repository information during code generation.

In terms of correctness, we observe that Local Reuse Correctness is optimal for A^1 Generation (local-aware), Global Reuse Correctness is best for A^2 Generation (local-aware and

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

TABLE IV
COMPARISON OF THE OVERALL EFFECT OF THE FRAMEWORK WITH ABLATION BASELINES USING GPT-3.5-TURBO-16K FOR A^1-CODGEN, A^2-CODGEN, AND A^3-CODGEN CONFIGURATIONS, AND COMPARING A^3-CODGEN WITH GPT-3.5-TURBO-16K AND GPT-4O

Metrics		Vanilla GPT-3.5-Turbo-16k	A^1-CodGen (GPT-3.5-Turbo-16k)	A^2-CodGen (GPT-3.5-Turbo-16k)	A^3-CodGen (GPT-3.5-Turbo-16k)	A^3-CodGen (GPT-4o)
Local Reuse	Awareness (F1)	N/A	0.673	0.673	0.694	**0.718**
	Correctness (F1)	N/A	0.675	0.645	0.632	**0.879**
Global Reuse	Awareness (F1)	N/A	N/A	0.601	0.606	**0.701**
	Correctness (F1)	N/A	N/A	0.611	0.591	**0.676**
3rd-Party Library Reuse	Awareness (F1)	0.635	0.652	0.536	0.720	**0.898**
	Correctness (F1)	0.644	0.734	0.735	0.852	**0.894**
	Library Coverage	0.786	0.863	0.875	0.940	**0.976**
Avg Line of Code (LOC)		16.308	16.507	**12.598**	12.804	13.885

global-aware), and Third-Party Library Reuse Correctness is highest for A^3-CodGen. This suggests that increasing input knowledge may influence the model's performance in specific reuse scenarios. Nevertheless, A^3-CodGen exhibits only a slight difference from the baseline in local reuse and global reuse correctness (0.003-0.015). Despite this minor difference in correctness metrics and its superior performance in other aspects, A^3-CodGen demonstrates robust overall performance in local reuse, global reuse, and third-party library reuse.

Looking at the Library Coverage, A^3-CodGen achieves near-perfect performance with a score of 0.940, signifying its ability to effectively reuse pre-installed libraries and avoid compatibility issues such as "ModuleNotFoundError: No module named xxx." Furthermore, the framework consistently generates code with an optimal number of lines (LOC), indicating its capability to produce concise and efficient code.

By examining the last two columns (Table IV) of A^3-CodGen with GPT-3.5-Turbo-16k and GPT-4o, we observe significant improvements in local reuse, global reuse, and third-party library reuse for GPT-4o compared to GPT-3.5-Turbo-16k. This demonstrates that the same prompts achieve better performance as the foundation model's capabilities improve. Thus, our A^3-CodGen's performance enhances with the increased ability of the foundation model. More powerful foundation models in future will lead to even better results. However, the code generated by A^3-CodGen based on GPT-4o is slightly more complex, with an average LOC increase of one line, compared to GPT-3.5-Turbo-16k. This is because GPT-4o considers more logical coherence between code segments, resulting in slightly more complex code. Nonetheless, the generated code remains more concise than that produced by Vanilla GPT-based code generation.

Finding 4: *A^3-CodeGen framework excels in awaring three types of code repository knowledge, resulting in an excellent performance in terms of local reuse, global reuse, and third-party library reuse. Thanks to its outstanding reuse capabilities, our framework can generate concise code that aligns with the code repository's working environment. Additionally, A^3-CodGen's performance improves with the increased capabilities of the foundation model.*

E. RQ5: How Does A^3-CodGen Framework Compare to GitHub Copilot?

1) Motivation: Given advancements in AI-driven code generation, it is essential to compare our A^3-CodGen framework with leading tools like GitHub Copilot [9]. GitHub Copilot generates code by leveraging local and global contextual information from neighboring tabs,[2] similar to our context-aware mechanisms. If Copilot can outperform our approach, further development of our method may be unnecessary. However, if it cannot, our goal is to identify the limitations in Copilot's design for collecting context and explore how to enhance its code reuse capabilities using insights from our framework.

2) Experiment Methodology: For each function in the benchmark dataset, we locate the function within the repository using the repository name and fully qualified name, then remove the function code and the corresponding import statements. We provide the function's comment and definition to trigger GitHub Copilot to generate the code by pressing the 'tab' key. To assist Copilot in generating third-party library imports, we add a comment *#generate import statements if necessary.* If Copilot makes no suggestion, it indicates no import statements are needed. We retain all necessary local functions that could be reused within the function. Finally, we used the same labeling method mentioned in Section IV-C to label the generated code. The GitHub Copilot plugin version in this experiment is 1.5.15.6094, and we disable the auto-update in settings. We compare A^3-CodGen with GitHub CoPilot in terms of local reuse, global reuse, and third-party library reuse. We use the *RepoEval* dataset mentioned in Section IV-B2.

3) Result Analysis: To evaluate the statistical significance of the differences between A^3-CodGen and CoPilot, we conduct Welch's t-test [79] with a significance threshold of $p < 0.05$. The p-values for local reuse awareness and correctness are 0.0696 and 0.5590, indicating no significant difference. For global reuse awareness and correctness, the p-values are 0.0144 and 1.46e-06, and for third-party library reuse awareness, correctness, and coverage, the p-values are 1.39e-11, 0.0231, and 0.0176, indicating significant differences.

Fig. 5 compares GitHub Copilot and A^3-CodGen. For local reuse, GitHub CoPilot performs excellently with an F1 score of 0.790 for local reuse awareness, while A^3-CodGen achieves an F1 score of 0.718. This is because CoPilot reviews all working

[2]https://github.blog/ai-and-ml/github-copilot/how-github-copilot-is-getting-better-at-understanding-your-code/

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

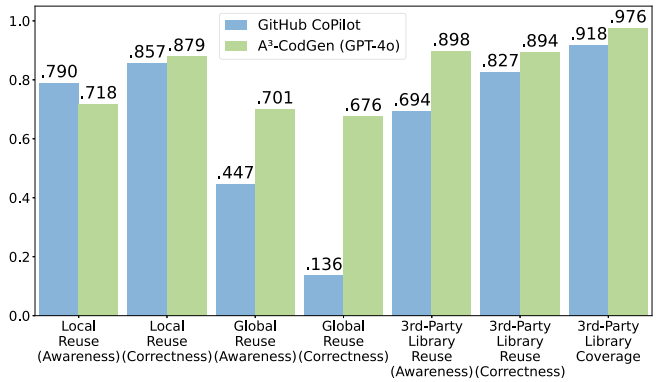

Fig. 5. Comparison between GitHub CoPilot and A^3-CodGen.

context details (including function definitions, function code, and descriptions) when generating the code. In terms of local reuse correctness, A^3-CodGen reaches an F1 score of 0.879, slightly better than GitHub CoPilot's 0.857, due to our structured approach to local knowledge.

However, for global reuse awareness and correctness, CoPilot's performance is suboptimal, with F1 scores of 0.447 and 0.136, respectively. CoPilot's workflow, although capable of utilizing neighboring tabs' context, struggles with global function calls as necessary global functions might exist in unopened tabs. Opening all repository tabs is impractical, and CoPilot does not structure information for completion, leading to chaotic context. In contrast, A^3-CodGen retrieves relevant global functions and structures them, facilitating easier and more accurate reuse. For third-party library reuse, A^3-CodGen outperforms CoPilot in awareness, correctness, and coverage. A^3-CodGen helps the model to be aware of all pre-installed libraries, whereas GitHub CoPilot randomly uses third-party libraries. While CoPilot excels in local information reuse, it lacks in global functions and third-party libraries reuse.

Our method has potential to enhance Copilot by using knowledge retrieval to fetch relevant global functions and library information from the repository, enabling repository-aware code generation. For instance, Copilot's limitations can be mitigated by incorporating interpreter information from the compiler to help Copilot become aware of third-party libraries. Additionally, using global function retrieval methods can assist Copilot in reusing functions from other code files, not just from neighboring tabs. Structuring the information before passing it to Copilot, rather than providing all information together, may also improve performance.

Finding 5: *GitHub CoPilot excels in local information reuse, but its global and third-party library reuse is suboptimal. A^3-CodGen performs comparably to CoPilot in local reuse while significantly outperforming in global and third-party library reuse due to structured knowledge retrieval. Our A^3-CodGen can potentially enhance AI-assistant tools like GitHub CoPilot for better code generation.*

VI. Discussion

A. New Paradigm for Code Generation Tasks

Code generation tasks have traditionally focused on automating code production for specific requirements, with advancements like GitHub CoPilot and ChatGPT mastering this task. However, real-world code generation extends beyond meeting requirements; it requires generating code aligned with project-specific needs and leveraging existing knowledge for improved readability, maintainability, and practicability. Therefore, we hope this work sheds a light on this emerging but significant code generation topic: generating context-aware code that is compatible with the working environment.

B. Knowledge Engineering for Code Generation

LLMs are sensitive to prompts and well-designed prompts are decisive in obtaining high-quality results [80], [81], [82]. Either unorganized or too lengthy prompts are impractical. Instead, it is advisable to extract semantic-enriched knowledge from the code repository to guide the model to directional generate code. However, how to selectively fuse the different types of knowledge presents a challenge, potentially impacting the model's awareness of different knowledge types. In Section V-A3, we find that providing diverse information does not necessarily improve model performance. Meanwhile, in section V-D, we find that providing the model with global awareness and third-party library information unexpectedly enhances its ability to reuse local functions. This underscores the interaction of diverse code knowledge types and stresses the significance of efficiently mining and utilizing code knowledge in future studies.

C. Dataset Construction for Context Aware Code Generation

1) Benchmark Representativeness: Experiments use 29 meticulously selected Python code repositories, emphasizing high quality indicated by stars and branches, while avoiding data leaks. These repositories collectively represent the Python programming ecosystem. Labeling all functions is highly labor-intensive. Therefore, as described in Section IV-B2, we perform sampling to ensure the estimated accuracy within the sample remains within a 0.05 error range while including functions from each code repository to ensure representativeness, with an average of 13.2 functions per repository.

2) Benchmark Construction for Different Contexts: In this paper, we validate the model's awareness of local functions, global functions, and third-party libraries. Future studies could include a more diverse range of repositories to test the framework's effectiveness in different contexts, such as awareness of classes, inherited functions, and variables. The dataset construction method can follow the approach in Section IV-B2, where we first label functions to create a benchmark, then evaluate the generated code by predicting labels, and finally calculate reuse awareness and correctness.

In the future, we will maintain and update this dataset, providing the community with a valuable resource for evaluating repository-aware code generation.

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

D. Threats to Validity

The threats to validity of this paper lies in the following aspects.

1) Subjective Labeling: Our study involves manual labeling, introducing subjectivity in annotations. We mitigate this by engaging two professional developers for labeling, resolving any instances with ambiguous labels to establish a reliable ground truth. The dataset is public for community validation [19].

2) Library Dependencies Control: Due to the complexity of library dependencies, constructing all 29 repository library dependencies is challenging. Thus, we do not consider the library version for A^3 code generation in our experiments. However, users can feasibly extract third-party libraries and generate code using the 'pip list' command or by checking the Python Interpreter in their environment to specify specific versions of libraries, as mentioned in Section III-A2.

3) Generalizing the A3-CodGen Framework to Other Programming Languages: While the experiments in this paper are conducted on Python code repositories, our A^3-CodGen framework is not limited to Python. To adapt it to another language, such as Java, developers need to use language-specific program analysis tools (e.g., Spoon [83] for Java) for the *Repository Knowledge Base Construction*. This involves extracting relevant repository knowledge, such as signatures and source code. Other steps, including *repository-aware knowledge retrieval* and A^3 *code generation*, remain unchanged. Although constructing the knowledge base may require additional engineering effort, it does not involve new scientific innovations. It simply requires following our outlined approach using the appropriate program analysis tools for the specific language.

VII. Related Work

Code generation tasks is about creating source code from provided natural language descriptions or requirements, which has been a longstanding focal point in SE research.

Repository-Level Code Completion. Repository-level code completion is gaining traction over traditional function-level completion [36], [38], [84], [85]. Shrivastava et al. generated example-specific prompts using a prompt proposal classifier [84] and further proposed *RepoFusion* to incorporate relevant repository context by training code models [38]. In addition, Ding et al. [36] proposed a cross-file context finder tool, named *CCFinder*, which constructs a project context graph and retrieves the most relevant cross-file context based on local import statements. *RepoCoder* [85] retrieved similar code snippets from the repository as part of a prompt and proposes an iterative retrieval-generation paradigm. This novel strategy bridged the gap between the retrieved context and the intended target through an interactive process.

Retrieval Augmented Code Generation. Retrieval Augmented Code Generation (RACG) [82], [86], [87], [88] represents an emerging and promising area of research that combines the strengths of retrieval-based methods with code generation techniques. Hayati et al. [82] introduced *RECODE*, the first model to integrate information retrieval into code generation. Parvez et al. [87] proposed the *REDCODER*, which utilizes retrieval to enhance the input of the generation model, aiming to maximize the utilization of additional information. Drain et al. [86] retrieve template functions from GitHub and relevant answers from Stack Overflow to improve code generation. Zhou et al. [88] proposed Docprompting, which retrieve relevant APIs from documents and add them into prompts to improve code generation. Zan et al. [89] proposed CodeGenAPI, which retrieve private APIs from documents to improve code generation. The APIRetriever module of CodeGenAPI and our approach both focus on API recommendation. However, CodeGenAPI directly retrieves APIs from documentation and cannot effectively handle high-level tasks that require combinations of APIs.

Code Completion Evaluation. LLM-based code completion and program synthesis methods provide unprecedented convenience, however researcher also question the actual performance under the mirage from different perspectives [16], [34], [58], [59], [60]. Yuan et al. evaluated 10 open-source instructed LLMs on four representative code comprehension and generation tasks [58]. Liu et al. proposed *EvalPlus*, a code synthesis benchmarking framework, to rigorously evaluate the functional correctness of LLM-synthesized code in function level [59]. In a more challenging code generation scenario, i.e., class-level code generation, Du et al. manually constructed the first class-level code generation benchmark *ClassEval* of 100 class-level Python code generation tasks [34]. In addition, Ouyang et al. conducted an empirical study to demonstrate that non-determinism in code generation tasks [60]. Yu et al. proposed a benchmark named *CoderEval*, which can be used to assess the performance of models against pragmatic code generation beyond just generating standalone functions [16].

VIII. Conclusion and Future Work

Code generation within code repositories often necessitates comprehensive knowledge of the entire repository context. Existing techniques often lack such a holistic understanding of code repository knowledge, which is crucial for effective code generation. In this paper, we introduce the A^3-CodGen framework, which is designed to construct, mine, and unify three essential types of code repository knowledge: local-aware, global-aware, and third-party-library-aware knowledge. To validate the effectiveness of these knowledge types, we conducted benchmark experiments and identified optimal configurations for their utilization. The results demonstrate that the A^3-CodGen framework significantly enhances the ability of LLMs to fully utilize local, global, and third-party library information. This results in the generation of concise, high-quality code. In the future, we plan to expand the applicability of our framework to multiple programming languages. Furthermore, we aim to enhance our knowledge utilization methods to enable LLMs to become more aware of various types of code knowledge within code repositories.

Acknowledgment

The authors would like to thank the anonymous reviewers who have provided insightful and constructive comments on this paper.

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

REFERENCES

[1] A. Svyatkovskiy, S. K. Deng, S. Fu, and N. Sundaresan, "Intellicode compose: Code generation using transformer," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2020, pp. 1433–1443.

[2] Y. Li et al., "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[3] M. Puschel et al., "SPIRAL: Code generation for DSP transforms," *Proc. IEEE*, vol. 93, no. 2, pp. 232–275, Feb. 2005.

[4] B. Chen et al., "CODET: Code generation with generated tests," 2022, *arXiv:2207.10397*.

[5] Z. Sun, Q. Zhu, Y. Xiong, Y. Sun, L. Mou, and L. Zhang, "TreeGen: A tree-based transformer architecture for code generation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, 2020, pp. 8984–8991.

[6] M.N.M. García, I.R. Román, F. J. G. Peñalvo, and M. T. Bonilla, "An association rule mining method for estimating the impact of project management policies on software quality, development time and effort," *Expert Syst. Appl.*, vol. 34, no. 1, pp. 522–529, 2008.

[7] M. A. Cusumano and D. B. Yoffie, "Software development on internet time," *Computer*, vol. 32, no. 10, pp. 60–69, 1999.

[8] OpenAI, "ChatGPT," 2023. [Online]. Available: https://openai.com/blog/chatgpt.

[9] "GitHub copilot. your AI pair programmer," [Online]. Available: https://copilot.github.com/

[10] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.

[11] M. Ciniselli, L. Pascarella, E. Aghajani, S. Scalabrino, R. Oliveto, and G. Bavota, "Source code recommender systems," 2023, *arXiv:2302.04098*.

[12] M. Liu, T. Yang, Y. Lou, X. Du, Y. Wang, and X. Peng, "Codegen4libs: A two-stage approach for library-oriented code generation,".

[13] D. Zan et al., "Cert: Continual pre-training on sketches for library-oriented code generation," 2022, *arXiv:2206.06888*.

[14] T. Li, E. B. Neundorfer, Y. Agarwal, and J. I. Hong, "Honeysuckle: Annotation-guided code generation of in-app privacy notices," in *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 5, no. 3, pp. 1–27, 2021.

[15] D. Zan et al., "Private-library-oriented code generation with large language models," 2023, *arXiv:2307.15370*.

[16] H. Yu et al., "CoderEval: A benchmark of pragmatic code generation with generative pre-trained models," 2023, *arXiv:2302.00288*.

[17] Y. Zhuang et al., "Efficiently measuring the cognitive ability of LLMS: An adaptive testing perspective," 2023, *arXiv:2306.10512*.

[18] D. Gifany, I. Amiri, M. Ranjbar, and J. Ali, "Logic codes generation and transmission using an encoding-decoding system," *Int. J. Adv. Eng. & Technol.*, vol. 5, no. 2, pp. 37–45, 2013.

[19] "Our dataset and repository," Accessed: Oct. 10, 2024. [Online]. Available: https://github.com/Dianshu-Liao/AAA-Code-Generation-Framework-for-Code-Repository-Local-Aware-Global-Aware-Third-Party-Aware

[20] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.

[21] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.

[22] D. Guo et al., "Graphcodebert: Pre-training code representations with data flow," 2020. *arXiv:2009.08366*.

[23] E. Nijkamp et al., "CodeGen: An open large language model for code with multi-turn program synthesis," in *Proc. Int. Conf. Learn. Representations*, 2022.

[24] D. Fried et al., "InCoder: A generative model for code infilling and synthesis," 2022, *arXiv:2204.05999*.

[25] S. Lu et al., "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," 2021, *arXiv:2102.04664*.

[26] W. Takerngsaksiri, C. K. Tantithamthavorn, and Y. Li, "Syntax-aware on-the-fly code completion," 2022, *arXiv:2211.04673*.

[27] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proc. 6th ACM SIGPLAN Int. Symp. Mach. Program.*, 2022.

[28] Q. Zheng et al., "CodeGeeX: A pre-trained model for code generation with multilingual evaluations on HumanEval-X," 2023, *arXiv:2303.17568*.

[29] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Proc. Chi Conf. Human Factors Comput. Syst. Extended Abstracts*, 2022, pp. 1–7.

[30] "Code completion in pycharm," 2023. Accessed: Oct. 10, 2024. [Online]. Available: https://www.jetbrains.com/help/pycharm/auto-completing-code.html

[31] Y. Wang, W. Wang, S. R. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.

[32] "GPT-4 technical report," OpenAI. 2023. Accessed: Oct. 10, 2024. [Online]. Available: https://cdn.openai.com/papers/gpt-4.pdf

[33] F. Shi et al., "Large language models can be easily distracted by irrelevant context," 2023, *arXiv:2302.00093*.

[34] X. Du et al., "ClassEval: A manually-crafted benchmark for evaluating LLMS on class-level code generation," 2023, *arXiv:2308.01861*.

[35] Y. Chang et al., "A survey on evaluation of large language models," 2023, *arXiv:2307.03109*.

[36] Y. Ding et al., "CoCoMIC: Code completion by jointly modeling in-file and cross-file context," 2022, *arXiv:2212.10007*.

[37] H. Pei, J. Zhao, L. Lausen, S. Zha, and G. Karypis, "Better context makes better code language models: A case study on function call argument completion," 2023, *arXiv:2306.00381*.

[38] D. Shrivastava, D. Kocetkov, H. Vries, D. Bahdanau, and T. Scholak, "RepoFusion: Training code models to understand your repository," 2023, *arXiv:2306.10998*.

[39] H. Stefan et al., "Code reuse in open source software," *Manag. Sci.*, vol. 54, pp. 180–193, 2008.

[40] J. Turner, "The structure of modular program," *Commun. ACM*, vol. 23, no. 5, pp. 272–277, 1980.

[41] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 783–794.

[42] Q. Huang et al., "SE factual knowledge in frozen giant code model: A study on FQN and its retrieval," 2022, *arXiv:2212.08221*.

[43] T. B. Brown et al., "Language models are few-shot learners," 2020, *arXiv:2005.14165v4*.

[44] S. Ye, H. Hwang, S. Yang, H. Yun, Y. Kim, and M. Seo, "In-context instruction learning," 2023, *arXiv:2302.14691*.

[45] "GPT-3.5-turbo-16k," OpenAI. 2023. Accessed: Sep. 2023. [Online]. Available: https://platform.openai.com/docs/models/gpt-3-5

[46] Q. Huang, Y. Sun, Z. Xing, M. Yu, X. Xu, and Q. Lu, "API entity and relation joint extraction from text via dynamic prompt-tuned language model," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 1, pp. 1–25, 2023.

[47] Q. Huang, Z. Yuan, Z. Xing, X. Xu, L. Zhu, and Q. Lu, "Prompt-tuned code language model as a neural knowledge base for type inference in statically-typed partial code," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–13.

[48] D. Zan et al., "CERT: Continual pre-training on sketches for library-oriented code generation," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 2369–2375.

[49] M. Liu, T. Yang, Y. Lou, X. Du, Y. Wang, and X. Peng, "CodeGen4Libs: A two-stage approach for library-oriented code generation," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2023, pp. 434–445.

[50] Z. Ma, S. An, B. Xie, and Z. Lin, "Compositional API recommendation for library-oriented code generation," in *Proc. IEEE/ACM 32nd Int. Conf. Program Comprehension (ICPC)*, 2024, pp. 87–98.

[51] X. Ren, X. Ye, D. Zhao, Z. Xing, and X. Yang, "From misuse to mastery: Enhancing code generation with knowledge-driven AI chaining," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2023, pp. 976–987.

[52] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.

[53] X. Wang, J. Wei, and D. Schuurmans, "Quoc le," in *Self-Consistency Improves Chain of Thought Reasoning in Language* Models, E. Chi, S. Narang, A. Chowdhery, and D. Zhou Eds., 2022, *arXiv:2203.11171*.

[54] Q. Lyu et al., "Faithful chain-of-thought reasoning," 2023, *arXiv:2301.13379*.

[55] OpenAI. Accessed: Sep. 2023. [Online]. Available: https://platform.openai.com/docs/guides/embeddings/what-are-embeddings

[56] A. Gao, "Vec2Vec: A compact neural network approach for transforming text embeddings with high fidelity," 2023, *arXiv:2306.12689*.

[57] M. Gü et al., "Jina embeddings 2: 8192-token general-purpose text embeddings for long documents," 2023, *arXiv:2310.19923*.

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric

[58] Z. Yuan, J. Liu, Q. Zi, M. Liu, X. Peng, and Y. Lou, "Evaluating instruction-tuned large language models on code comprehension and generation," 2023, *arXiv:2308.01240*.

[59] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? Rigorous evaluation of large language models for code generation," 2023, *arXiv:2305.01210*.

[60] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "LLM is like a box of chocolates: the non-determinism of chatGPT in code generation," 2023, *arXiv:2308.02828*.

[61] A. Matton et al., "On leakage of code generation evaluation datasets," 2024, *arXiv:2407.07565*.

[62] Z. Yang, Z. Zhao, C. Wang, J. Shi, D. Kim, D. Han, and D. Lo, "Unveiling memorization in code models," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 867–879.

[63] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyvanyk, and G. Bavota, "Using pre-trained models to boost code review automation," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, 2022, pp. 2291–2302.

[64] A. Mastropaolo, L. Pascarella, and G. Bavota, "Using deep learning to generate complete log statements," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, 2022, pp. 2279–2290.

[65] M. Alfadel, D. E. Costa, and E. Shihab, "Empirical analysis of security vulnerabilities in python packages," *Empirical Softw. Eng.*, vol. 28, 2021, pp. 1–37.

[66] "Pycallgraph," 2023. Accessed: Sep. 2023. [Online]. Available: https://github.com/gak/pycallgraph

[67] "Pyan," 2023. Accessed: Sep. 2023. [Online]. Available: https://github.com/davidfraser/pyan

[68] R. P. Singh and N. S. Mangat, "Elements of survey sampling," vol. 15, Springer Science & Business Media, 2013.

[69] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2002, pp. 311–318.

[70] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2004, pp. 74–81.

[71] S. Banerjee and A. Lavie, "METEOR: An automatic metric for mt evaluation with improved correlation with human judgments," in *Proc. ACL Workshop Intrinsic Extrinsic Eval.,* 2005, pp. 228–231.

[72] J. Landis and G. G. Koch, "An application of hierarchical kappa-type statistics in the assessment of majority agreement among multiple observers," *Biometrics*, vol. 33, no. 2, 1977, pp. 363–74.

[73] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[74] M. Zhang, O. Press, W. Merrill, A. Liu, and N. A. Smith, "How language model hallucinations can snowball," 2023, *arXiv:2305.13534*.

[75] S. Roller et al., "Recipes for building an open-domain chatbot," in *Proc. 16th Conf. Eur. Chapter Assoc. Comput. Linguistics: Main Volume*, Association for Computational Linguistics, Apr. 2021, pp. 300–325.

[76] M. Si et al., "A solution toward transparent and practical AI regulation: Privacy nutrition labels for open-source generative AI-based applications," 2024, *arXiv:2407.15407*.

[77] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Proc. Neural Inf. Process. Syst.*, 2023.

[78] "Gpt-4o," OpenAI. 2024. Accessed: Oct. 10, 2024. Accessed: Oct. 10, 2024. [Online]. Available: https://openai.com/index/hello-gpt-4o/

[79] W. Bl, "The generalization of 'student's' problem when several different population varlances are involved," *Biometrika*, vol. 34, no. 1–2, pp. 28–35, Jan. 1947.

[80] C. Yang, Y. Liu, and C. Yin, "Recent advances in intelligent source code generation: A survey on natural language based studies," *Entropy*, vol. 23, no. 9, pp. 1–27, 2021.

[81] J. Shin and J. Nam, "A survey of automatic code generation fromnatural language," *J. Inf. Process. Syst.*, vol. 17, no. 3, pp. 537–555, 2021.

[82] S. A. Hayati, l. O. Raphaë, P. Avvaru, P. Yin, A. Tomasic, and G. Neubig, "Retrieval-based neural code generation," 2018, *arXiv:1808.10025*.

[83] R. Pawlak, M. Monperrus, N. Petitprez, C. Noguera, and L. Seinturier, "Spoon: A library for implementing analyses and transformations of Java source code," *Softw.: Pract. Exp.*, vol. 46, no. 9, pp. 1155–1179, 2015.

[84] H. Disha Shrivastava and D. Tarlow, Repository-level prompt generation for large language models of code. ArXiv:abs/2206.12839, 2022.

[85] B. Fengji Zhang et al., "RepoCoder: Repository-level code completion through iterative retrieval and generation," 2023, *arXiv:2303.12570*.

[86] D. Drain, C. Hu, C. Wu, M. Breslav, and N. Sundaresan, "Generating code with the help of retrieved template functions and stack overflow answers," 2021, *arXiv:2104.05310*.

[87] M. R. Parvez, W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Retrieval augmented code generation and summarization," 2021, *arXiv:2108.11601*.

[88] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, "DocPrompting: Generating code by retrieving the docs," in *Proc. Int. Conf. Learn. Representations*, 2022.

[89] D. Zan, B. Chen, Z. Lin, B. Guan, Y. Wang, and J.-G. Lou, "When language model meets private library," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2022.

Licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 29,2026 at 20:05:12 UTC from IEEE Xplore. Restric